

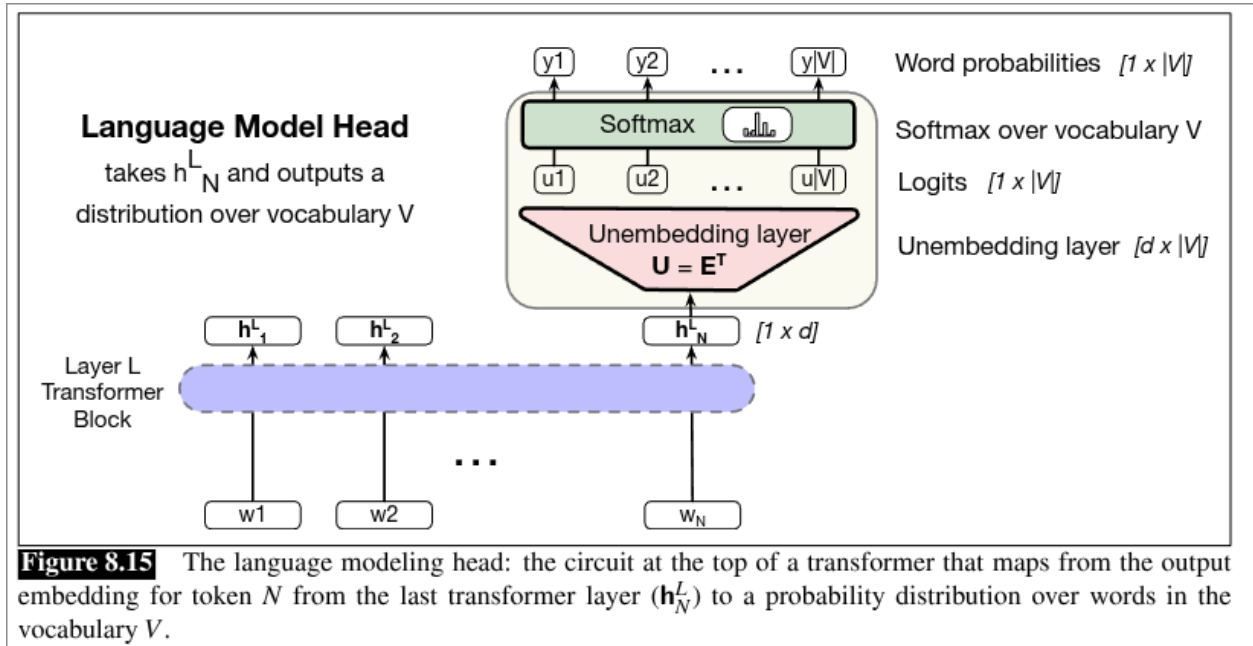
8.5 The language modeling head

Phần này giới thiệu language modeling head, tức là thành phần cuối cùng được đặt lên trên transformer để model thực sự làm được bài toán language modeling. Nói ngắn gọn, transformer trước đó chủ yếu tạo ra các hidden representation cho từng token trong chuỗi, còn language modeling head là phần biến hidden representation cuối cùng đó thành dự đoán cho từ tiếp theo. Đây cũng là bước nối từ “biểu diễn bên trong model” sang “xác suất trên vocabulary”.

Về bản chất, language model luôn là một bộ dự đoán từ tiếp theo. Khi cho một context, model sẽ gán xác suất cho mọi từ có thể xuất hiện sau đó, từ đó tạo ra một phân phối xác suất trên toàn bộ vocabulary. Điểm khác biệt là với các mô hình cũ như n-gram thì context khá ngắn và cố định, còn với transformer thì context có thể rất dài, bằng đúng context window của nó. Vì vậy, language modeling head của transformer cũng đang làm đúng bài toán quen thuộc của language modeling, chỉ là nó dựa trên một biểu diễn ngữ cảnh mạnh hơn và lớn hơn rất nhiều.

Cách hoạt động của language modeling head có thể tóm tắt như sau:

- Nó lấy output của token cuối cùng ở layer cuối, tức hidden state tại vị trí N .
- Hidden state này được đưa qua một linear layer để chiếu từ không gian d chiều sang một vector logits có kích thước bằng số từ trong vocabulary.
- Thực tế, ma trận linear này thường không học riêng hoàn toàn, mà hay được tied với embedding matrix ở đầu vào.
- Cụ thể hơn, người ta thường dùng E^T , tức transpose của embedding matrix E , để ánh xạ từ hidden representation quay trở lại vocabulary space.
- Vì E^T đang thực hiện phép ánh xạ ngược từ embedding về từ vựng, nó còn được gọi là unembedding layer.
- Sau đó, softmax sẽ biến logits thành một phân phối xác suất thực sự trên toàn bộ vocabulary.



Ý tưởng weight tying được áp dụng ở đây, ta sẽ tận dụng cùng một ma trận E cho hai việc ngược nhau: lúc vào thì map token sang embedding, lúc ra thì dùng E^T để map embedding quay lại không gian từ vựng. Điều này giúp mô hình tái sử dụng biểu diễn đã học thay vì phải học một ma trận đầu ra hoàn toàn tách biệt. Nói cách khác, embedding layer và unembedding layer không phải là hai phần hoàn toàn độc lập, mà có thể được buộc liên kết với nhau để model học nhất quán hơn.

Khi đã có phân phối xác suất trên vocabulary, model có thể dùng nó theo hai cách chính. Một là dùng để tính xác suất của văn bản, tức đánh giá một chuỗi từ có hợp lý không. Hai là dùng để generate token tiếp theo, đây mới là ứng dụng quan trọng nhất trong thực tế. Từ phân phối đó, ta có thể chọn từ xác suất cao nhất theo greedy decoding, hoặc dùng các phương pháp sampling như top-k hay top-p để sinh văn bản đa dạng hơn (được nói đến ở phần sau).

Có lưu ý là, kiểu transformer dùng cho causal language modeling như vậy thường được gọi là decoder-only model. Tên này xuất phát từ việc nó tương ứng với phần decoder của kiến trúc encoder-decoder trong transformer gốc. Nói cách khác, mô hình language model hiện đại thường

chỉ giữ lại nhánh phù hợp cho sinh token tuần tự, thay vì dùng toàn bộ kiến trúc encoder-decoder như trong các bài toán kiểu machine translation.

8.6 more on sampling

Những phương pháp lấy mẫu được nói đến trong chương này cho phép ta thực hiện sự đánh đổi giữa hai yếu tố quan trọng là quality và diversity của văn bản được sinh ra.

- Những phương pháp chọn trực tiếp từ có xác suất cao nhất thường tạo ra câu trả lời rõ ràng, mạch lạc và bám sát ngữ cảnh hơn, nên nhìn chung sẽ cho cảm giác chính xác và an toàn hơn. Tuy nhiên, điểm yếu của chúng là đầu ra rất dễ bị lặp ý, lặp từ, hoặc rơi vào các cách diễn đạt quen thuộc, từ đó làm giảm tính đa dạng.
- Ngược lại, những phương pháp nghiêng nhiều hơn về các từ có xác suất trung bình thường cho mô hình nhiều không gian lựa chọn hơn, nên đầu ra có xu hướng phong phú, linh hoạt và sáng tạo hơn. Dù vậy, vì không luôn ưu tiên phương án có xác suất cao nhất, các cách này cũng có thể làm nội dung kém ổn định hơn, đôi khi ít chính xác hoặc kém tự nhiên hơn.
- Nói ngắn gọn, sampling không chỉ là chọn từ tiếp theo, mà còn là cơ chế điều chỉnh phong cách đầu ra của mô hình giữa hai cực: an toàn, chính xác và dễ đoán ở một phía, với đa dạng, mới mẻ và sáng tạo ở phía còn lại.

8.6.1 Top-k sampling

Đây là một dạng tổng quát hóa của greedy decoding. Thay vì luôn chọn duy nhất từ có xác suất cao nhất ở mỗi bước, top-k sampling sẽ giới hạn không gian lựa chọn vào k từ có xác suất cao nhất trước, sau đó chuẩn hóa lại xác suất trên tập nhỏ này, rồi mới sample ra từ cuối cùng dựa trên phân phối mới. Nói cách khác, mô hình vẫn chỉ xét những ứng viên mạnh nhất, nhưng không còn bị ép phải chọn đúng từ đúng đầu như greedy decoding.

Một cách formal hơn:

- Chọn trước một giá trị k, đây là một hyperparameter do ta đặt.
- Dùng language model để tính xác suất của từng từ trong vocabulary V với context hiện tại.
- Sắp xếp các từ theo xác suất của chúng, rồi chỉ giữ lại k từ có xác suất cao nhất.
- Renormalize lại các score của k từ này để thu được một phân phối xác suất mới hợp lệ trên chính tập k từ đó.
- Cuối cùng, sample từ tiếp theo từ phân phối xác suất mới này.

Khi $k = 1$, phương pháp này trở thành greedy decoding vì lúc đó chỉ còn đúng một lựa chọn. Còn khi chọn k lớn hơn, mô hình đôi khi có thể chọn một từ không phải cao xác suất nhất nhưng vẫn là một phương án rất hợp lý trong ngữ cảnh. Nhờ vậy, văn bản sinh ra thường đa dạng và bớt lặp hơn, trong khi chất lượng vẫn được giữ lại ở mức khá tốt vì mô hình chỉ lấy mẫu trong nhóm những từ có khả năng cao nhất.

8.6.2 Nucleus / top-p sampling

Một vấn đề của top-k sampling là giá trị k phải được cố định từ trước, trong khi hình dạng của phân phối xác suất của mô hình lại thay đổi theo từng context. Có những lúc phân phối rất nhọn, nên chỉ cần lấy top 10 từ là đã bao phủ gần như phần lớn xác suất rồi. Nhưng cũng có những lúc phân phối phẳng hơn, khi đó cùng một giá trị k = 10 lại chỉ bao phủ được một phần nhỏ của toàn bộ phân phối. Vì vậy, một giá trị k cố định có thể phù hợp ở context này nhưng lại khá cứng và kém linh hoạt ở context khác.

Top-p hay nucleus sampling ra đời để xử lý vấn đề đó. Thay vì giữ lại đúng k từ có xác suất cao nhất, phương pháp này sẽ giữ lại một tập từ nhỏ nhất sao cho tổng xác suất cộng dồn của chúng đạt tới ngưỡng p đã chọn trước. Bản chất, mục tiêu của top-p vẫn khá giống top-k: loại bỏ các từ có xác suất quá thấp để tránh sinh ra những lựa chọn vô lý, nhưng vẫn giữ lại đủ mức ngẫu nhiên để đầu ra đa dạng hơn. Điểm khác là top-p xét theo xác suất tích lũy của cả nhóm từ, chứ không xét theo số lượng từ cố định, nên nó thường thích nghi tốt hơn với các context khác nhau.

Quy trình của top-p hay nucleus sampling nhìn chung khá giống top-k. Mô hình vẫn tính xác suất cho toàn bộ vocabulary trước, nhưng ở bước chọn tập ứng viên, ta dùng công thức (8.48) để lấy ra tập từ sao cho tổng xác suất của chúng vừa đủ đạt ngưỡng p. Sau đó, các xác suất trong tập này sẽ được renormalize lại để tạo thành một phân phối mới hợp lệ, rồi từ tiếp theo sẽ được sample từ phân phối đó. Nói cách khác, số lượng từ được giữ lại trong nucleus có thể thay đổi theo từng bước sinh, và chính sự thay đổi linh hoạt đó là ưu điểm lớn của top-p so với top-k.

$$\sum_{w \in V(p)} P(w|\mathbf{w}_{<t}) \geq p. \quad (8.48)$$

8.7 Training

Quá trình training của language model đã được mô tả ở chương trước. Nhắc lại, các large language model thường được huấn luyện bằng cross-entropy loss, còn gọi là negative log likelihood loss. Ở thời điểm t , cross-entropy loss chính là negative log-probability mà model gán cho từ đúng ở vị trí tiếp theo, tức là $-\log(p(w_{t+1}))$. Nói cách khác, nếu model gán xác suất càng cao cho từ đúng thì loss càng nhỏ, còn nếu đoán sai hoặc quá thiếu tự tin thì loss sẽ lớn hơn.

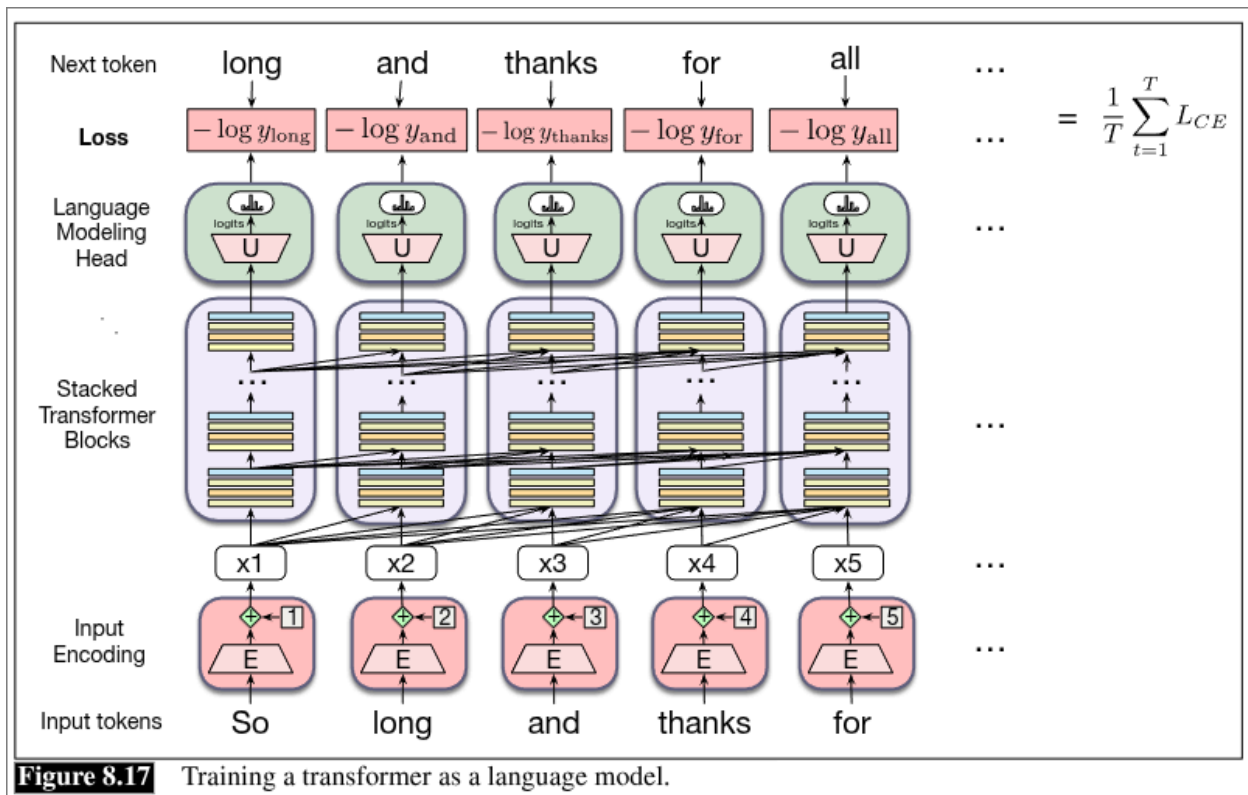
Figure 8.17 minh họa quá trình training của transformer:

- Ở mỗi bước, transformer sẽ output ra một phân phối xác suất trên toàn bộ vocabulary V cho vị trí đang xét.
- Trong training, xác suất mà model gán cho từ đúng sẽ được dùng để tính cross-entropy loss tại từng vị trí trong sequence.
- Loss của một training sequence thường được lấy bằng trung bình các cross-entropy loss của toàn bộ các từ trong câu hoặc trong đoạn đó.
- Sau khi tính loss, các trọng số của model sẽ được cập nhật thông qua backpropagation và gradient descent như trong các mô hình neural network thông thường.

Một số điểm thú vị của transformer:

- Mỗi training item trong sequence có thể được xử lý song song, vì model tạo output cho mọi vị trí gần như cùng lúc thay vì phải chạy tuần tự từng bước như các kiến trúc cũ hơn. Đây cũng là một trong những lý do quan trọng khiến transformer huấn luyện rất hiệu quả trên phần cứng hiện đại.
- Thông thường, các model lớn sẽ được train bằng cách tận dụng tối đa context window. Nếu một tài liệu không đủ dài để lấp đầy cửa sổ ngữ cảnh này, người ta thường ghép nhiều tài liệu lại với nhau và ngăn cách chúng bằng các ký hiệu end-of-file đặc biệt.

- Batch size dùng trong gradient descent của các mô hình rất lớn cũng thường được tính theo số token thay vì chỉ theo số câu hay số văn bản. Với các mô hình cực lớn như GPT-3, batch size có thể lên tới hàng triệu token, ví dụ bản lớn nhất được huấn luyện với batch size khoảng 3.2 triệu token. Điều này giúp model học từ một lượng ngữ cảnh rất lớn trong mỗi bước cập nhật.



8.8 Dealing with scale

Các LLM hiện đại có kích thước cực kỳ lớn, và ngay trong phần 8.8 của tài liệu cũng có đưa ra những ví dụ rất rõ về quy mô này. Chẳng hạn, sách

nêu ví dụ Llama 3.1 405B Instruct có tới 405 tỉ tham số, 126 layer, kích thước mô hình 16,384 và 128 attention heads, đồng thời được huấn luyện trên 15.6 terabyte token văn bản. Chính vì quy mô khổng lồ như vậy nên đã xuất hiện rất nhiều hướng nghiên cứu nhằm tìm hiểu cách LLM scale lên như thế nào, cả về mặt hiệu năng lẫn chi phí huấn luyện. Đặc biệt, một mối quan tâm rất lớn là làm sao cài đặt, huấn luyện và suy luận các mô hình này một cách hiệu quả khi tài nguyên tính toán, bộ nhớ và thời gian đều có giới hạn. Đây cũng là lý do phần tiếp theo tập trung vào các khái niệm như scaling laws và các kỹ thuật giúp mô hình vận hành hiệu quả hơn trong thực tế.

8.8.1 Scaling laws

Performance của LLM đã được cho thấy là phụ thuộc chủ yếu vào 3 yếu tố:

- Model size, tức số lượng tham số của mô hình, thường không tính embedding parameters.
- Dataset size, tức lượng dữ liệu dùng để huấn luyện.
- Lượng compute được dùng cho training, nghĩa là tổng tài nguyên tính toán bỏ ra trong quá trình huấn luyện. Điều này có thể tăng lên bằng cách tăng số layer, tăng độ dài context, train trên nhiều dữ liệu hơn, hoặc train qua nhiều iteration hơn.

Mối quan hệ giữa những yếu tố này và performance được gọi là scaling law. Nói đơn giản hơn, scaling law cố gắng mô tả khi ta tăng kích thước mô hình, tăng dữ liệu hoặc tăng compute thì loss sẽ giảm như thế nào, và mức cải thiện đó có tuân theo một quy luật đủ ổn định để ta dự đoán trước hay không. Trong phần này, performance chủ yếu được nhìn thông qua loss của mô hình.

Giả sử như Kaplan et al. 2020 đưa ra mối quan hệ giữa:

- Hàm loss L
- Số parameters N, không tính embedding parameters
- Dataset size D
- Lượng compute C

$$L(N) = \left(\frac{N_c}{N} \right)^{\alpha_N} \quad (8.49)$$

$$L(D) = \left(\frac{D_c}{D} \right)^{\alpha_D} \quad (8.50)$$

$$L(C) = \left(\frac{C_c}{C} \right)^{\alpha_C} \quad (8.51)$$

Số lượng tham số N có thể được ước lượng gần đúng từ các thành phần kiến trúc của transformer. Trong đó d là chiều input-output của mô hình, d_attn là chiều của self-attention, còn d_ff là chiều của lớp feedforward. Ý chính ở đây là số tham số tăng rất nhanh khi ta tăng chiều rộng mô hình và số layer, nên việc scale model không chỉ là “thêm một ít” tài nguyên mà thường kéo theo mức tăng rất lớn về bộ nhớ và compute.

$$\begin{aligned} N &\approx 2 d n_{\text{layer}} (2 d_{\text{attn}} + d_{\text{ff}}) \\ &\approx 12 n_{\text{layer}} d^2 \\ &\quad (\text{assuming } d_{\text{attn}} = d_{\text{ff}}/4 = d) \end{aligned} \quad (8.52)$$

Các tham số trong công thức (8.49), (8.50), (8.51) phụ thuộc vào kiến trúc transformer cụ thể, cách tokenization và cả vocabulary size. Vì vậy, scaling laws thường không quá tập trung vào giá trị tuyệt đối của từng hằng số, mà tập trung nhiều hơn vào xu hướng chung giữa các đại lượng này với hàm loss. Nói cách khác, điều quan trọng nhất không phải là đúng chính xác từng con số trong mọi mô hình, mà là hiểu được dạng quan hệ tổng quát: khi tăng N, D hoặc C thì loss thường giảm theo một quy luật kiểu power law.

Scaling laws khá hữu dụng khi:

- Ta cần quyết định nên train model như thế nào để đạt một mức performance nhất định.
- Ta muốn dự đoán loss sẽ thay đổi ra sao nếu tăng thêm data hoặc tăng kích cỡ model.
- Ta muốn ước lượng cần thêm bao nhiêu data khi scale model lên, thay vì chỉ tăng tham số một cách mù quáng.
- Ta muốn dùng các thí nghiệm nhỏ hơn hoặc giai đoạn đầu của training để dự đoán hành vi của một mô hình lớn hơn, từ đó tiết kiệm chi phí thử nghiệm.

Phần này thực ra có thể mở rộng thêm khá nhiều, vì scaling laws không chỉ là một tập công thức, mà còn là một cách nhìn về việc thiết kế và huấn luyện LLM. Thay vì chỉ hỏi “mô hình lớn hơn có tốt hơn không”, scaling laws giúp ta hỏi chính xác hơn: lớn hơn bao nhiêu thì đáng, cần bao nhiêu dữ liệu để đi kèm, và compute nên được phân bổ như thế nào để việc scale mang lại hiệu quả thực sự. Đây cũng là lý do scaling laws rất quan trọng trong thực tế, đặc biệt khi chi phí huấn luyện LLM là cực kỳ lớn.

8.8.2 KV Cache

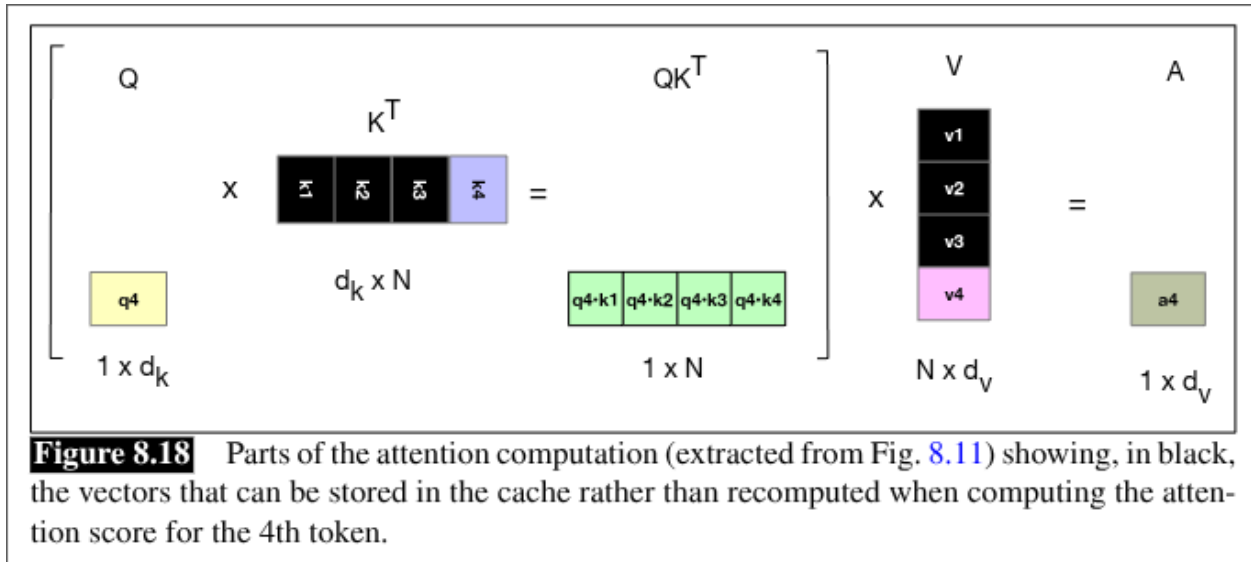
Attention vector có thể được tính toán song song trong quá trình training bằng công thức ma trận quen thuộc của self-attention, tức là tính Q , K , V cho cả sequence rồi thực hiện phép nhân ma trận để thu được attention cho tất cả token cùng lúc. Đây là cách tính rất hiệu quả trong training vì toàn bộ các token trong sequence đã có sẵn, nên mô hình có thể xử lý chúng song song thay vì phải đi từng bước một. Công thức này cũng chính là dạng song song của attention đã được trình bày trước đó trong phần 8.3 và được nhắc lại ở mục 8.8.2.

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} \right) \mathbf{V} \quad (8.53)$$

Tuy nhiên, cách tính này không thể được dùng nguyên xi cho inference. Lý do là ở inference, mô hình phải generate từng token một theo kiểu autoregressive. Khi một token mới được sinh ra, ta sẽ tính query, key và value của token đó bằng cách nhân với các ma trận W^Q , W^K , W^V . Nếu mỗi lần sinh token mới mà lại áp dụng lại cách tính như trong training cho toàn bộ chuỗi, thì ta sẽ phải tính lại các token cũ $j < i$, dù các giá trị đó thực ra đã được tính ở những bước trước. Điều này gây lãng phí tính toán rất lớn.

Vì vậy, trong inference người ta dùng KV cache. Ý tưởng là sau khi tính key và value cho các token trước đó, ta lưu chúng lại trong bộ nhớ cache. Đến khi sinh token mới, mô hình chỉ cần tính thêm query, key, value cho token mới này, rồi lấy các key và value cũ trực tiếp từ cache thay vì phải tính lại từ đầu. Nhờ đó, inference trở nên hiệu quả hơn rất nhiều, đặc biệt khi chuỗi đầu vào ngày càng dài.

Figure 8.18 minh họa đúng ý tưởng này trong inference. Hình cho thấy khi tính attention cho một token mới, một phần các vector cần thiết đã có thể lấy trực tiếp từ cache, và chỉ những thành phần liên quan đến token mới sinh ra mới cần được tính thêm. Nói ngắn gọn, KV cache không thay đổi bản chất của attention, mà chỉ tránh việc lặp lại những phép tính đã có từ các bước trước.



8.8.3 Parameter Efficient Fine Tuning

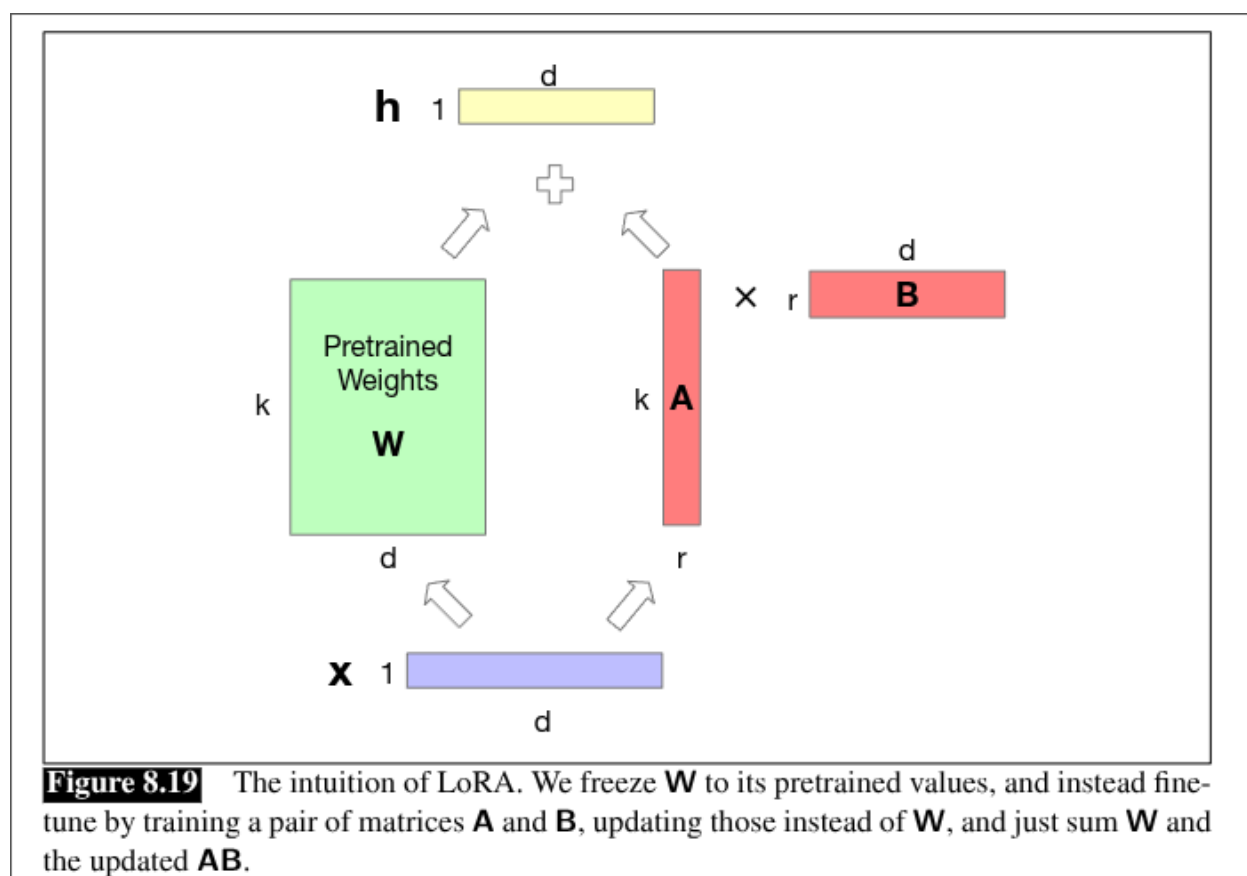
Một việc rất phổ biến trong language modeling là lấy một pretrained model có sẵn rồi bổ sung cho nó thêm kiến thức về một domain mới bằng cách finetune trên dữ liệu của domain đó. Cách làm này rất tự nhiên vì model ban đầu đã có năng lực ngôn ngữ tổng quát, còn finetuning giúp nó thích nghi tốt hơn với thuật ngữ, phong cách viết, và các mẫu dữ liệu chuyên biệt của domain mới. Tuy nhiên, finetuning lại có thể rất khó với các LLM lớn, vì bản chất chúng có số lượng tham số cực kỳ lớn, và mỗi lần chạy batch gradient descent đều phải backpropagate qua rất nhiều layer rất nặng. Điều này làm cho việc finetune trở nên tốn tài nguyên tính toán, tốn bộ nhớ và tốn thời gian. Chính vì thế mới xuất hiện các phương pháp chỉ finetune một phần nhỏ tham số thay vì thay đổi toàn bộ mô hình, và nhóm phương pháp này được gọi là parameter-efficient fine tuning, hay PEFT.

Trong phần này ta chủ yếu nói về LoRA, tức Low-Rank Adaptation. Ý tưởng chính của LoRA là transformer có rất nhiều dense layer thực hiện phép nhân ma trận, ví dụ như các ma trận WQ , WK , WV , WO trong

attention. Thay vì finetune trực tiếp toàn bộ các ma trận lớn này, LoRA sẽ đóng băng chúng và chỉ học một phần hiệu chỉnh có hạng thấp hơn, tức một low-rank approximation. Cách nhìn này dựa trên trực giác rằng khi thích nghi sang một domain mới, ta không nhất thiết phải thay đổi toàn bộ không gian tham số, mà chỉ cần một cập nhật nhỏ nhưng có cấu trúc phù hợp là đã đủ để mô hình điều chỉnh hành vi.

Giả sử ta có một ma trận W có kích thước $[k \times d]$. Nếu finetune theo cách thông thường, ta sẽ học một ma trận cập nhật ΔW cũng có cùng kích thước $[k \times d]$. Nhưng trong LoRA, thay vì học trực tiếp ΔW , ta tạo ra hai ma trận nhỏ hơn là A và B , trong đó A có kích thước $[k \times r]$, B có kích thước $[r \times d]$, với r là một hyperparameter nhỏ hơn rất nhiều so với k và d . Khi đó tích AB sẽ đóng vai trò xấp xỉ cho phần cập nhật ΔW , nên thay vì thay W thành $W + \Delta W$, ta dùng $W + AB$. Theo công thức (8.54) trong tài liệu, nếu forward ban đầu là $h = xW$ thì với LoRA nó trở thành $h = xW + xAB$. Như vậy, thay vì phải finetune cả một ma trận lớn, ta chỉ cần finetune hai ma trận nhỏ hơn nhiều là A và B .

Điểm mạnh lớn nhất của LoRA là nó giảm mạnh chi phí finetuning cho các mô hình lớn, vì phần lớn tham số của model không cần tính gradient nữa. Ta chỉ cập nhật A và B , nên yêu cầu về bộ nhớ và compute giảm đi đáng kể. Hơn nữa, LoRA còn có tính modular khá cao: sau khi finetune cho từng domain khác nhau, các cặp ma trận A, B này có thể được lưu riêng và gắn vào mô hình gốc khi cần. Vì AB có cùng kích thước với W , phần cập nhật có thể được cộng thẳng vào trọng số pretrained, nên gần như không làm tăng thêm thời gian inference. Đây là lý do LoRA rất phù hợp trong thực tế khi ta muốn tái sử dụng cùng một base model cho nhiều domain khác nhau với chi phí thấp. Trong phiên bản gốc, LoRA chủ yếu được áp dụng cho các ma trận trong attention, dù về sau đã có nhiều biến thể mở rộng hơn.



8.9 Interpreting the Transformer

Làm sao các mô hình transformer-based lại có thể làm tốt các tác vụ ngôn ngữ đến như vậy? Đây là câu hỏi trung tâm của interpretability, hay cụ thể hơn trong nhiều ngữ cảnh là mechanistic interpretability, một hướng nghiên cứu tập trung vào việc hiểu rõ bên trong transformer thực sự đang làm gì thay vì chỉ nhìn vào đầu vào và đầu ra của nó. Mục tiêu của hướng này là phân tích xem những thành phần nào trong mô hình đang lưu trữ thông tin, xử lý mẫu ngôn ngữ, hay phối hợp với nhau để tạo ra hành vi thông minh mà ta quan sát được. Trong phần này, ta sẽ tập trung vào hai khía cạnh đã được nghiên cứu khá kỹ của transformer interpretability, tức là hai nơi mà

các nhà nghiên cứu đã có tương đối nhiều bằng chứng và công cụ để bắt đầu giải thích cơ chế hoạt động bên trong mô hình.

8.9.1 In-Context Learning and Induction Heads

Ta có thể hiểu quá trình prompting là khác với quá trình pretraining. Học qua pretraining nghĩa là model thật sự cập nhật parameters của nó bằng gradient descent để làm giảm hàm loss. Còn prompting với ví dụ thì không hề thay đổi trọng số của model; thay vào đó, các ví dụ trong prompt giúp model tạm thời suy ra cách thực hiện task ngay trong lúc nó đang xử lý chuỗi đầu vào. Nói cách khác, model không “học” theo nghĩa cập nhật weights, mà học theo nghĩa tận dụng thông tin có sẵn trong context để hành xử tốt hơn ở inference time.

Thuật ngữ “in-context learning” lần đầu được Brown et al. (2020) đề xuất khi giới thiệu GPT-3, để chỉ hiện tượng model học từ chính prompt của nó. Vì vậy, in-context learning có thể hiểu là việc language model học cách làm một task mới, dự đoán token tốt hơn, hay nói chung là giảm loss trong lúc chạy forward pass ở inference, mà không cần bất kỳ gradient-based update nào trên parameters của model. Đây là một điểm rất đặc biệt, vì nó cho thấy một phần hành vi “học” của transformer có thể xuất hiện ngay trong quá trình suy luận chứ không chỉ trong training.

In-context learning hoạt động như thế nào thì hiện tại ta vẫn chưa biết chắc. Tuy vậy, đã có một số giả thuyết khá thú vị, trong đó nổi bật là giả thuyết dựa trên induction heads. Induction heads là tên của một circuit, tức một thành phần trừu tượng bên trong network, và induction head circuit là một phần của quá trình attention trong transformer. Chúng được phát hiện khi người ta quan sát các mini language model rất nhỏ, chỉ có khoảng 1–2 attention heads, để dễ nhìn rõ từng cơ chế hơn.

Công dụng chính của induction head là hỗ trợ dự đoán các pattern lặp lại trong sequence. Ví dụ, nếu model thấy một mẫu kiểu ABA...A, thì induction head có xu hướng đoán rằng token tiếp theo sẽ là B. Cơ chế trực giác ở đây là nó thực hiện một dạng prefix matching: khi gặp token A ở hiện tại, nó tìm ngược về trước một lần xuất hiện trước đó của A, rồi “copy” token từng đi sau A ở lần trước, tức B, để tăng xác suất B xuất hiện tiếp theo ở hiện tại. Đây là một hành vi rất thú vị vì nó cho thấy attention head không chỉ nhìn ngữ cảnh một cách mơ hồ, mà đôi khi còn thực hiện những phép truy hồi mẫu khá cụ thể.

Nói chung, đây vẫn là một hiện tượng quan sát được và là một manh mối quan trọng để phần nào soi sáng cách transformer hoạt động, chứ chưa phải là một lời giải hoàn chỉnh cho toàn bộ in-context learning. Tuy nhiên, giả thuyết này cũng không phải hoàn toàn cảm tính, vì tài liệu có nhắc đến các nghiên cứu ablation cho thấy khi làm yếu hoặc loại bỏ các induction heads thì khả năng in-context learning của model giảm đi rõ rệt. Điều đó không chứng minh rằng induction heads giải thích toàn bộ hiện tượng, nhưng ít nhất cho thấy chúng có liên quan thực sự đến cách model học từ prompt.



Tiếp theo, pdf có nói thêm về một số work tiếp tục đào sâu vào behaviour của induction heads để xem liệu chúng có thật sự liên quan đến in-context learning hay không. Một hướng kiểm tra khá quan trọng là tìm ra các attention heads có hành vi giống induction heads, rồi ablate chúng, tức là làm cho chúng ngừng hoạt động bằng cách zero-out output của chúng. Kết quả được nhắc tới trong phần này cho thấy khi các induction heads bị

zero-out như vậy thì khả năng in-context learning của model giảm đi rõ rệt, đặc biệt là khả năng học từ các ví dụ demonstrations trong prompt trở nên tệ hơn. Nói cách khác, đây chưa phải là bằng chứng kết luận hoàn toàn cơ chế của in-context learning, nhưng nó là một dấu hiệu khá mạnh cho thấy induction heads thực sự có ảnh hưởng nhân quả nhất định đến hiện tượng in-context learning, chứ không chỉ đơn giản là một pattern thú vị được quan sát ngẫu nhiên.

8.9.2 Logit lens

Logit lens đơn giản là một phương pháp để visualize các lớp bên trong transformer. Ý tưởng của nó là thử nhìn vào một vector ở giữa mạng và hỏi rằng: nếu coi vector này như thể nó đã là biểu diễn cuối cùng trước khi đi vào language modeling head, thì nó đang “ngiên” về những từ nào trong vocabulary. Đây là một cách khá trực quan để soi vào các representation trung gian của model, thay vì chỉ nhìn output cuối cùng.

Cụ thể, nó lấy một vector ở bất kỳ layer nào của transformer, nhân vector đó với unembedding layer E^T , rồi tính softmax để thu được một phân phối xác suất trên các từ. Từ phân phối này, ta có thể xem thử vector đó có vẻ đang represent những token nào mạnh nhất. Nói cách khác, logit lens biến một hidden state trung gian thành một “dự đoán tạm thời” trên vocabulary để ta dễ quan sát hơn.

Tất nhiên, cách nhìn này chỉ là một phép xấp xỉ chứ không hoàn hảo. Lý do là các hidden state ở giữa mạng vốn không được train với mục tiêu phải trực tiếp ánh xạ ra token distribution như lớp cuối cùng, nên đôi khi logit lens có thể cho ra một bức tranh chưa thật sự chính xác. Dù vậy, nó vẫn là một công cụ rất hữu ích để giúp ta hình dung các layer bên trong transformer có thể đang mã hóa loại thông tin gì trong quá trình xử lý.